

Robotics Software Engineer Hiring Assignment: Adaptive Navigation and Conflict-Aware Path Planning (Logistics Focus)

Scenario Introduction: You are a senior Robotics Software Engineer tasked with commissioning a new fleet of **Autonomous Mobile Robots (AMRs)** for a highly-automated logistics warehouse. The core challenge is to maximize order fulfillment throughput by implementing a resilient, cooperative navigation stack that can handle **heterogeneous robot types** and **dynamic congestion**. Your solution must feature cooperative mapping, velocity control based on payload, and a rock-solid, low-latency safety system capable of overriding the central planner in real-time unsafe scenarios.

1. Navigation Setup and Environment

- **Robot Models (Heterogeneous Fleet):**
 - **AMR-1 (Mapper/Lead):** Focus on accurate global localization and path planning. **Higher payload capacity.**
 - **AMR-2 (Scout/Follower):** Focus on speed and dynamic obstacle avoidance. **Lower payload capacity, higher acceleration limits.**
 - **Simulation Environment (Gazebo):** A large, multi-level structure (ramps required for testing terrain cost) with complex, initially unknown aisle layouts. The environment includes static storage racks and frequent, randomly moving **dynamic obstacles** (simulated pedestrian traffic or third-party robots).
-

2. Core Mapping and Global Path Planning

The candidate must implement the following core navigation capabilities:

1. **Cooperative Global SLAM & Map Fusion:**
 - Implement a system where both AMR-1 and AMR-2 contribute to a **single, unified global occupancy grid map** of the entire warehouse.
 - **Challenge (Selective Mapping):** AMR-1's map updates (publishing to the merged map) must be configured to prioritize **unexplored boundaries** and reduce the update frequency for areas it has repeatedly traversed (simulating resource-efficient **selective iteration** to refine global map performance).
2. **Adaptive Global Planner Implementation:**
 - **Goal Setting:** Set concurrent navigation goals for both robots (e.g., AMR-1 to "Heavy Storage," AMR-2 to "Packing Bay 4").

- **Ramp/Slope Planning:** The Global Planner must use a custom cost function (or a tuned configuration) that correctly calculates the traversability cost of **sloped surfaces/ramps**, minimizing their use unless they are the only viable path to the goal.
-

3. Multi-Robot Local Path Planning and Safety Control

This section focuses on the real-time movement execution and safety overrides in a congested environment.

1. Dynamic Velocity and Motion Smoothing:

- Implement a custom local controller or a wrapper that enforces **Motion Smoothing** where the robot's acceleration and jerk are smoothly limited based on its **dynamic state** (e.g., current speed and whether it is carrying a maximum payload).
- The smoothing must be physically appropriate for the robot's type (e.g., AMR-1, the heavier unit, must have lower acceleration limits than AMR-2, the lighter scout).

2. Conflict-Aware Trajectory Generation (MAPF Element):

- The Local Planner for **each robot** must be configured to consume the **projected trajectories of the other robot** (AMR-1 and AMR-2) to avoid predicted conflicts.
- In inevitable conflict scenarios (e.g., robots approaching a narrow intersection simultaneously), the Traffic Control Node must enforce a **pre-defined yielding protocol** (e.g., the lighter AMR-2 always yields to the heavier, mission-critical AMR-1) by commanding a temporary, controlled stop.

3. Safety System Override (Pre-emptive Avoidance):

- Implement a dedicated safety node that monitors the local obstacle detection results.
 - If the system detects *any* obstacle closer than a **critical, speed-dependent threshold** ($d_{safe} = k \cdot v^2 + d_{min}$), it must issue a low-level, high-priority **immediate halt command** that **bypasses (overrides)** the navigation stack's current velocity command. This directly tests the fundamental "system override in unsafe scenarios" experience.
-

4. Sensor/Data Flow Integration and Standards

1. Validated Data Consumption (HAL Alternative):

- Instead of a classic HAL, implement a **Board Support Package (BSP)** style validation routine. The navigation stack nodes must consume sensor data (LiDAR, IMU) only after it passes through this routine.

- **Validation Requirement:** The routine must log a warning if the **IMU data** for either robot shows angular velocity that exceeds a physically plausible limit for the robot model (demonstrating the validation toolkit experience).
- 2. **Code Quality and Scalability:**
 - All multi-robot management and control code (velocity smoothing, traffic control) must be cleanly **namespaced** and implemented with a reusable class structure to allow the fleet size to be expanded to ten or more robots by changing a minimal number of configuration parameters.

5. Deployment and Code Quality (Testing Yocto, Refactoring, Codebase)

1. **Environment Setup and Build:**
 - The candidate must provide a complete, well-documented **README** detailing the steps to launch the entire simulation, navigation, and custom control stack.
 - The code must be organized into a proper ROS/ROS 2 workspace using CMake/Colcon and include necessary dependencies for a clean build.
 - To submit a screenshare video explaining the various integrities of the problem statement.
2. **Code Refactoring and Standards:**
 - All custom nodes developed must adhere to a strict style guide (e.g., Google C++ Style or standard Python PEP 8).
 - **Deliverable:** The candidate must identify one area of the standard ROS Navigation or Gazebo launch files they integrated and propose a refactoring plan (e.g., splitting a monolithic launch file into modular components or converting a complex parameter file into a clean C++ struct/enum representation) and implement a small part of that refactoring.

Evaluation Criteria

Skill Area Tested	Relevant Experience Point	Success Metrics
-------------------	---------------------------	-----------------

SLAM & Global Planning	<i>Mapping its own environment</i>	Generates a complete, high-quality map; successfully navigates from base to rescue zone.
Safety & Override	<i>Obstacle detection, system override</i>	Robot correctly performs a controlled stop or evasive maneuver when an obstacle violates the safe distance constraint, overriding user input, Dynamic obstacles.
Motion Control	<i>Motion smoothing based on dynamic velocity</i>	Robot's movement is observably smooth; velocity profile correctly modulates based on environment type.
Custom Integration	<i>Developed HAL and validation toolkit</i>	A functional HAL class is used by the navigation node; the validation checks correctly run and log outputs for the IMU and Camera.
System Optimization	<i>Improved global mapping... selective iteration</i>	Demonstrated ability to programmatically restrict the map update area based on a defined condition.
Code Quality & Build	<i>Refactoring and minor functional updates</i>	Clean, well-commented, and modular code; workspace builds cleanly; provides a compelling refactoring demonstration.